

Codificação Para Front-end

08

2026

SENAI - CRTI

HTML Forms

Username:

Password:

Aula 3: Formulários e Validação Nativa

Dando continuidade ao nosso projeto de e-commerce, hoje vamos aprender a criar formulários estruturados e funcionais com validação nativa no HTML — um passo essencial para permitir que usuários se cadastrem, façam contato e realizem compras.

DESENVOLVIMENTO FRONT-END

Por que formulários são essenciais na web?



Cadastro de Usuários

Permitem criar contas, armazenar preferências e personalizar a experiência de compra no e-commerce.



Processo de Compra

Coletam endereço de entrega, dados de pagamento e confirmação de pedido — etapas críticas do funil de vendas.



Contato e Suporte

Facilitam a comunicação entre cliente e empresa, essencial para fidelização e resolução de problemas.



Segurança e Validação

Com validação adequada, garantem que os dados recebidos sejam corretos, seguros e úteis para o sistema.



Estrutura básica de um formulário HTML

Código fundamental

```
<form
  id="form-contact"
  action="/enviar"
  method="POST"
  novalidate>

  <fieldset>
    <legend>Dados de Contato</legend>

    <label for="input-name">
      Nome:
    </label>
    <input
      type="text"
      id="input-name"
      name="name"
      required>

  </fieldset>

  <button type="submit">
    Enviar
  </button>

</form>
```

<form>

Elemento raiz que agrupa todos os campos. O atributo `action` define para onde os dados vão e `method` define como (GET ou POST).

<fieldset> e <legend>

Agrupam campos relacionados visualmente e semanticamente, melhorando acessibilidade e organização.

<label>

Essencial para acessibilidade. O atributo `for` deve coincidir com o `id` do campo correspondente.

<button type="submit">

Aciona o envio do formulário. Use sempre com `type` explícito para evitar comportamentos inesperados.

Campos de entrada e suas aplicações no e-commerce

O HTML5 oferece mais de 20 tipos de `input`. Escolher o tipo correto melhora a experiência do usuário — especialmente em dispositivos móveis, onde o teclado se adapta automaticamente ao tipo de campo.

`type="text"`

Campo de texto livre. Usado para nome, endereço, cidade. Aceita qualquer caractere por padrão.

`type="email"`

Valida automaticamente o formato de e-mail. No celular, exibe teclado com @ e .com.

`type="password"`

Oculto o texto digitado. Essencial para campos de senha e confirmação de senha.

`type="number"`

Aceita apenas números. Permite configurar `min`, `max` e `step` para controlar o intervalo válido.

`type="tel"`

Campo de telefone. Abre teclado numérico no celular. Combine com `pattern` para validar o formato.

  **Desafio rápido:** Abra o formulário no celular e observe como o teclado muda entre os campos de email, número e texto!

Fieldset, Legend e boas práticas de estrutura

Em formulários longos — como o de cadastro de um e-commerce — é fundamental **agrupar campos relacionados**. O `<fieldset>` cria seções visuais e semânticas, enquanto o `<legend>` nomeia cada grupo.

```
<form id="form-register">

  <fieldset>
    <legend>Dados Pessoais</legend>
    <!-- nome, cpf, data nasc. -->
  </fieldset>

  <fieldset>
    <legend>Dados de Acesso</legend>
    <!-- email, senha -->
  </fieldset>

  <fieldset>
    <legend>Preferências</legend>
    <!-- interesses, newsletter -->
  </fieldset>

</form>
```

- 1 Melhora a leitura por leitores de tela
Tecnologias assistivas anunciam o grupo ao entrar em cada `fieldset`, contextualizando o usuário.
- 2 Facilita a estilização com CSS
Cada `fieldset` pode receber estilos distintos, criando seções visualmente separadas no formulário.
- 3 Organiza a lógica de validação
Facilita validar seção por seção, o que será muito útil quando avançarmos para JavaScript.

Select, Radio e Checkbox

`<select>` — Lista suspensa

Ideal quando há muitas opções predefinidas. Use para categoria de interesse, estado, país.

```
<select name="category"
id="input-category">
  <option
value="">Selecione...</option>
  <option
value="eletronicos">Eletrônicos</option>
  <option
value="moda">Moda</option>
</select>
```

`type="radio"` — Escolha única

Quando apenas uma opção pode ser selecionada. Use o mesmo `name` para agrupar as opções do mesmo grupo.

```
<input type="radio"
name="client-type"
id="radio-pf"
value="pf">
<label for="radio-pf">Pessoa Física</label>
<input type="radio"
name="client-type"
id="radio-pj"
value="pj">
<label for="radio-pj">Pessoa
Jurídica</label>
```

`type="checkbox"` — Múltipla escolha

Quando várias opções podem ser marcadas ao mesmo tempo. Ideal para preferências e interesses do usuário.

```
<input type="checkbox"
name="preferences"
id="check-offers"
value="offers">
<label for="check-offers">Receber
ofertas</label>

<input type="checkbox"
name="preferences"
id="check-news"
value="news">
<label for="check-news">Newsletter</label>
```

Atributos que protegem e orientam o usuário

Atributos de validação disponíveis

`required`

Campo obrigatório. O formulário não envia enquanto estiver vazio.

`minlength` / `maxlength`

Limita o número de caracteres aceitos. Ex: senha com no mínimo 8 caracteres.

`min` / `max`

Para inputs numéricos ou de data. Ex: idade Mínima de 18 anos.

`pattern`

Aceita expressão regular para validar formatos específicos como CEP ou CPF.

`placeholder`

Texto de dica dentro do campo. Não substitui o `label`!

Exemplo prático no e-commerce

```
<!-- Nome obrigatório -->
<input type="text"
  id="input-name"
  name="name"
  required
  minlength="3"
  placeholder="Seu nome completo">

<!-- E-mail com validação -->
<input type="email"
  id="input-email"
  name="email"
  required
  placeholder="seu@email.com">

<!-- Senha segura -->
<input type="password"
  id="input-password"
  name="password"
  required
  minlength="8"
  placeholder="Mínimo 8 caracteres">

<!-- Idade mínima -->
<input type="number"
  id="input-age"
  name="age"
  min="18"
  max="120"
  placeholder="Sua idade">
```



A validação nativa é uma primeira camada de proteção. **Sempre valide os dados também no servidor!**

UX em Formulários

Experiência do usuário: o que faz um formulário funcionar bem?

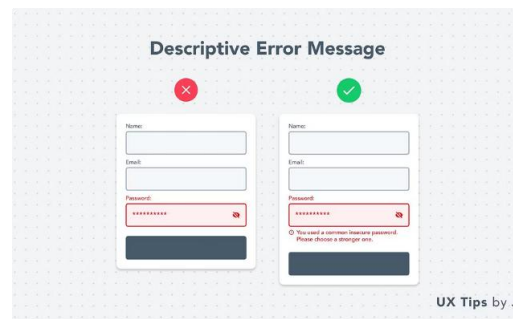
Um formulário tecnicamente correto pode ser uma péssima experiência se ignorar princípios básicos de usabilidade. No e-commerce, formulários mal projetados são uma das principais causas de abandono de carrinho.



The image shows a sample 'Application Form' with a dark teal background. It includes fields for 'First Name', 'Last Name', 'Date of birth' (with a date picker), 'Age' (with a dropdown), 'Gender' (with a dropdown), 'Email Address' (with a placeholder), 'Positions Available' (with radio buttons for Junior Developer, Mid-level Developer, and Senior Developer), 'Programming Languages' (with checkboxes for Java, JavaScript, and Python), 'Password', and 'Confirm Password'. There are 'Submit' and 'Reset' buttons at the bottom.

Labels sempre visíveis

Nunca use apenas **placeholder** como rótulo. Quando o usuário começa a digitar, o placeholder desaparece e ele perde a referência do campo.



The image shows a comparison of two error messages for a login form. The left side shows a form with a red 'X' icon and a generic error message. The right side shows a form with a green checkmark icon and a descriptive error message: 'You used a common insecure password. Please choose a stronger one.' The text 'UX Tips by Ji' is visible at the bottom right.

Mensagens de erro claras

Indique exatamente o que está errado e como corrigir. "Campo inválido" é frustrante; "Use pelo menos 8 caracteres" é útil.



Adaptar ao dispositivo

Usar os tipos corretos de input (**email**, **tel**, **number**) garante que o teclado certo apareça no celular, agilizando o preenchimento.

O que evitar ao criar formulários HTML

✗ Esquecer o atributo **for** no label

O **for** deve corresponder ao **id** do input. Sem isso, clicar no label não foca o campo — problema sério de usabilidade e acessibilidade.

✗ Usar **placeholder** como único rótulo

O placeholder desaparece ao digitar. Usuários com memória curta ou que revisam o formulário ficam sem saber o que o campo pede.

✗ Radio buttons sem mesmo **name**

Para que apenas uma opção seja selecionável no grupo, todos os radio buttons relacionados devem ter o mesmo valor no atributo **name**.

✗ Não usar **type** no botão

Um **<button>** sem **type** dentro de um formulário é tratado como **submit** por padrão, o que pode causar envios acidentais.

✗ Depender só da validação HTML

A validação nativa pode ser desabilitada pelo usuário via DevTools. Ela é uma boa experiência, mas nunca a única camada de segurança.

✗ Formulários sem **fieldset** em formulários longos

Sem agrupamento, campos soltos tornam o formulário confuso, especialmente para usuários com leitores de tela.

❓ **Desafio:** Remova o atributo **required** de um campo e tente enviar o formulário. O que acontece? E se remover o **type="email"**?

Evolução do Projeto: Formulário de Contato e Cadastro

Chegou a hora de colocar a mão no código! Você vai evoluir o e-commerce adicionando um formulário completo com todos os elementos aprendidos hoje.



Nome

Campo `text`, `required`, `minlength="3"`



E-mail

Campo `email` com validação automática de formato



Senha

Campo `password`, `minlength="8"`



Idade

Campo `number`, `min="18"`, `max="120"`



Categoria

Elemento `select` com opções de interesse



Preferências

Múltiplos `checkbox` para newsletter, ofertas etc.



Tipo de Cliente

Radio buttons: Pessoa Física ou Jurídica

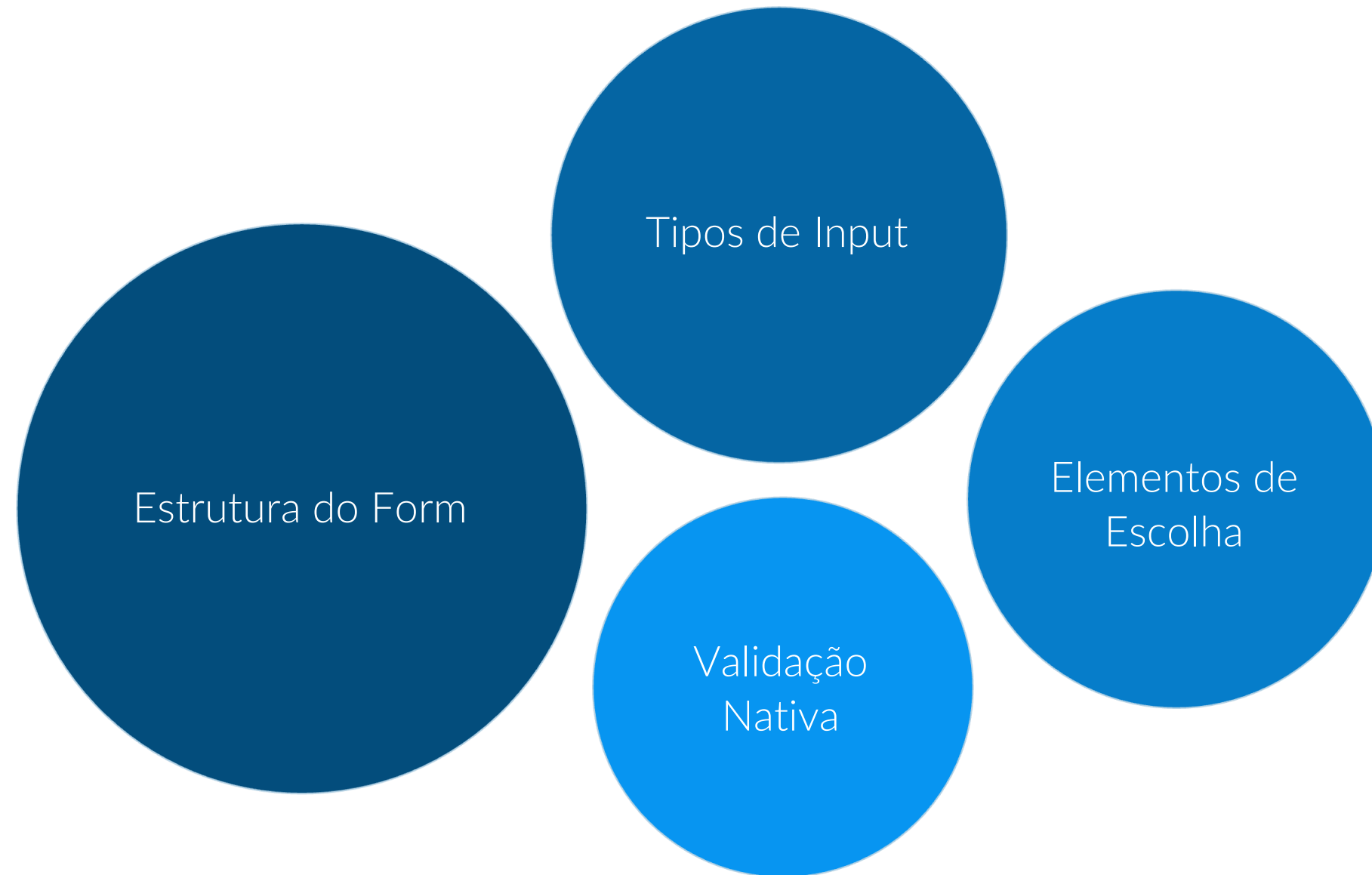


Mensagem

Elemento `textarea` para mensagem livre

Recapitulação

O que aprendemos hoje?



Nesta aula, evoluímos o e-commerce de um site estático para uma aplicação capaz de **coletar dados dos usuários** de forma estruturada, semântica e com validação básica já funcional no navegador. Cada elemento aprendido tem um papel claro na construção de interfaces reais.

Para se aprofundar

→ MDN Web Docs — Forms

developer.mozilla.org — Documentação completa sobre todos os elementos e atributos de formulários HTML. Referência essencial para qualquer desenvolvedor front-end.

→ W3C — HTML5 Constraint Validation

Especificação oficial da API de validação nativa do HTML5, explicando como os navegadores implementam cada regra de validação.

→ WebAIM — Accessible Forms

webaim.org — Guia de boas práticas de acessibilidade especificamente para formulários web, com exemplos práticos.



Pratique no seu editor

Abra o VS Code, crie um arquivo `form-test.html` e experimente adicionar e remover atributos para ver o efeito em tempo real no navegador.



Use o DevTools

No Chrome ou Firefox, inspecione os elementos do formulário criado. Observe como o navegador detecta os erros de validação antes mesmo de você clicar em enviar.



Teste no celular

Acesse seu formulário pelo celular e veja como o teclado muda para cada tipo de campo (`email`, `number`, `tel`).

Firjan SENAI

